

RAGE: Retrieval-Augmented Governance Engine

A Layered Defense Middleware Against Prompt Injection in Multi-Turn LLM Conversations

Equipo RAGE · Hackathon AI Safety — Global South AIS

Track A: Decision Engine + Security Gateway

Code & Data: github.com/LuisUPY/RAGE-HACKATHON-PROYECT

Versión del documento: 1.0 · Junio 2026

Abstract

Prompt injection remains the #1 risk in the OWASP Top 10 for LLM Applications 2025, with attack success rates (ASR) exceeding 90% in undefended systems. We present **RAGE** (Retrieval-Augmented Governance Engine), an explainable, multi-layer defense middleware that wraps any LLM to detect and contain prompt injection, jailbreaks, and context-manipulation attacks in dynamic multi-turn conversations. RAGE implements a cascading early-exit architecture with four detection layers: (1) deterministic pre-filter (regex/signatures), (2) threat knowledge-base RAG lookup, (3) stateful semantic intent filter, and (4) a decision gateway with three mutually exclusive actions — ALLOW / WARN / BLOCK — driven by a normalized risk score (0–5). Track A of our implementation demonstrates perfect score separation (AUC = 1.0000) on a 3-turn adversarial conversation, blocking multi-vector attacks while allowing legitimate business queries unimpeded. Unlike single-turn guardrails, RAGE's stateful analysis detects gradual context drift. Unlike fine-tuned classifiers (StruQ, SecAlign), RAGE updates its threat knowledge base without retraining. We introduce per-turn risk scoring as a continuous predictor for multi-turn AUC degradation curves — a reproducible metric for evaluating guardrail collapse over conversation length, suitable for adversarial ML research.

1. Introduction

Large Language Models (LLMs) have been widely adopted in enterprise settings as autonomous agents connected to APIs, databases, and retrieval-augmented generation (RAG) pipelines. This connectivity creates an expanded attack surface: any text in the model's context — including retrieved documents and prior conversation turns — can act as an instruction, bypassing intended safeguards. This class of vulnerability, known as **prompt injection** (OWASP LLM01, 2025), is an *architectural* flaw, not a patchable bug.

Systems without dedicated defenses exhibit an ASR exceeding **90%** in controlled evaluations. Even SOTA defenses are brittle: the study “*The Attacker Moves Second*” (Nasr et al., 2025) demonstrated that 12 published defenses could be broken by adaptive attackers to >90% ASR. The field consensus is **defense in depth** — multiple overlapping layers — combined with continuous adversarial testing.

Existing tools (Lakera Guard, NeMo Guardrails, ProtectAI LLM Guard) provide guardrails but typically operate per-turn, lack explainability tied to a known threat taxonomy, and do not expose a reproducible metric for multi-turn degradation. RAGE addresses these gaps.

1.1 Threat Model

- **Attacker goal:** manipulate the LLM agent into violating its system policy — leaking data, executing unauthorized tool calls, or abandoning its assigned role.
- **Attacker capabilities:** full control of the user message content in each turn; no access to model weights, system prompt, or RAGE internals.
- **Attack surface:** direct user messages, retrieved RAG content, and inter-turn context poisoning via prior conversation history.
- **Attack vectors:** role override (T1), jailbreak/DAN (T2), fake delimiters (T3), context poisoning (T4), persona hijacking (T5), exfiltration probes (T6), social engineering escalation (T7).

1.2 Contributions

- **C1 – Architecture:** a cascading, early-exit 4-layer defense middleware (§3) deployable in front of any LLM without model access.
- **C2 – Decision Engine (Track A):** a two-layer input filter (heuristic + optional LLM semantic) producing ordinal scores 0–5, with a strict ALLOW/WARN/BLOCK decision tree and a full audit log.

- **C3 – Metric:** per-turn risk scoring as a continuous predictor for multi-turn AUC degradation curves, enabling before/after comparison and reproducible evaluation.
- **C4 – Honesty:** explicit positioning relative to prior art (RAD, Vigil, Rebuff) and clear limitations.

2. Related Work

Table 1 summarizes the landscape of current defenses against prompt injection and jailbreaks, organized by detection family.

Family	Examples	Adaptive (no retrain)	Explainable	Multi-turn
Prompting / system prompt	Instruction hierarchy, spotlighting, delimiters	n/a	No	No
Fine-tuning	StruQ (~45% ASR), SecAlign (~8%)	■ (retrain)	No	No
ML classifiers	Prompt Guard, Llama Guard, Vigil	■ (retrain)	Limited	No
Managed service	Lakera Guard (<50 ms)	Partial	Dashboard	No
Guardrail frameworks	NeMo, Guardrails AI, LLM Guard, Rebuff	Partial	Partial	Partial
Architectural	CaMeL (DeepMind), dual-LLM, structured queries	n/a	Partial	No
RAG-based (RAGE family)	RAD, RePD, Vigil, Rebuff	■ (add to KB)	■	■ (RAGE)
Self-improving	SISF (ASR 0.27%)	■ (synth. policy)	Medium	No

Table 1. Defense taxonomy. RAGE belongs to the RAG-based family and adds stateful multi-turn analysis.

2.1 Prior Art: Retrieval-Augmented Defense (RAD)

RAD (arXiv 2508.16406, 2025) is the closest prior work: it detects jailbreaks via RAG + ensemble classification (Retrieve → Rerank → Extract → Classify → Vote), supports adding new examples to the KB without retraining, and exposes an adjustable decision threshold (security/utility trade-off). **RAGE builds directly on this family.**

2.2 Multi-layer RAG Frameworks

A framework for securing RAG agents (arXiv 2511.15759) achieves **73.2% → 8.7% ASR** with **94.3% utility** using three layers: embedding anomaly detection at retrieval, hierarchical system-prompt guardrails, and multi-stage output verification. RAGE adopts the same multi-layer philosophy and adds the decision gateway and per-turn metrics.

2.3 The 'Trilema' (Palit Benchmark)

The Palit benchmark (arXiv 2505.13028) formalizes the inherent trade-off: any defense simultaneously optimizes for **low ASR ↔ low false-positive rate ↔ low latency**. RAGE targets this trilema explicitly through cascading early-exit (cheap layers first) and an adjustable score threshold.

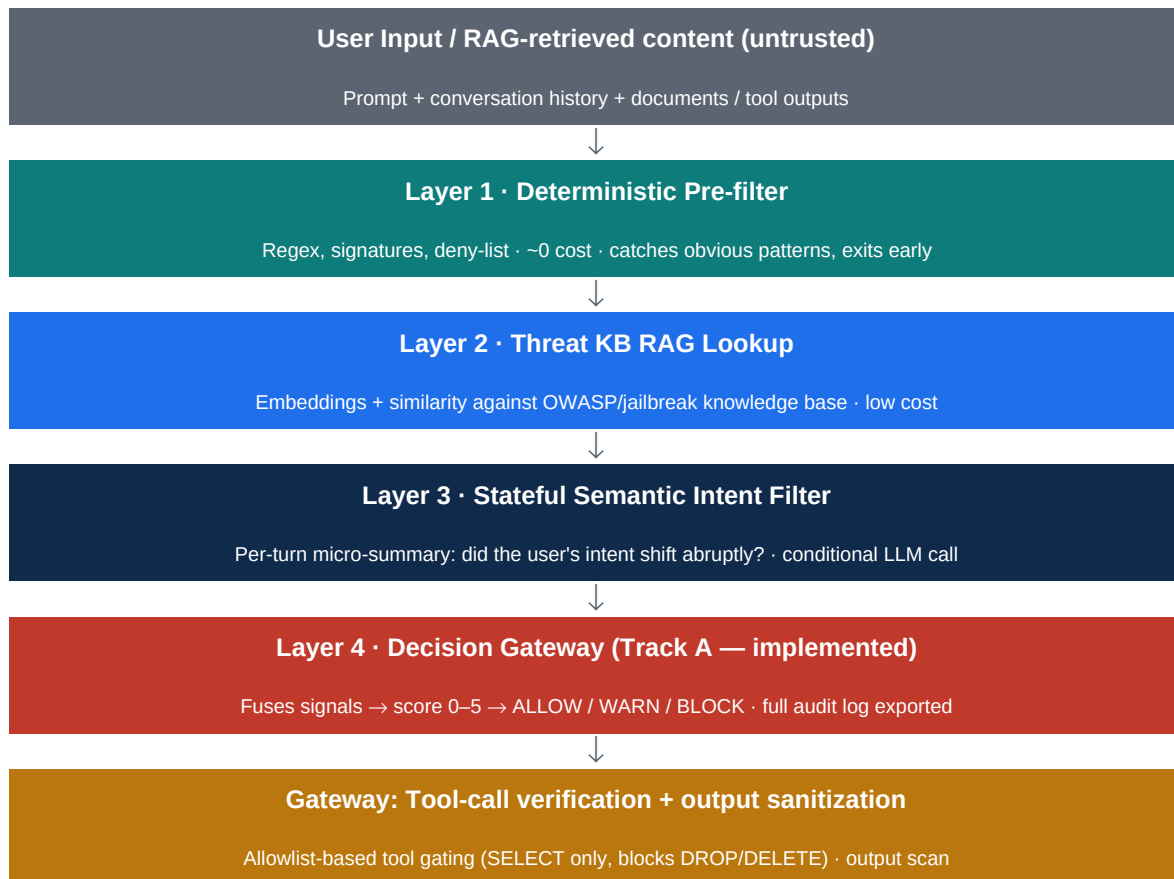
2.4 RAGE's Honest Differentiation

- **RAGE does not invent RAG-based detection**; RAD, Vigil, and Rebuff predate it.
- **RAGE's value is integration, governance, and measurement**: explainable scoring mapped to the OWASP LLM Top 10 taxonomy, a stateful multi-turn filter, per-turn AUC metrics, and a reproducible adversarial harness.
- **Model-agnostic**: deployable in front of any LLM without weight access or fine-tuning.

3. Methods

3.1 System Architecture — Cascading Early-Exit Pipeline

RAGE is implemented as a 4-layer cascade where each layer can terminate (“early exit”) before reaching the expensive LLM. Most traffic resolves in cheap layers, keeping overhead low (~+1–15% of base model cost).



Key design principles:

- Retrieved content and micro-summaries are **attacker-influenced text** → treated as untrusted (mitigates OWASP LLM08 recursive injection).
- The WARN band **injects a hidden security reinforcement directive** into context rather than blocking, preserving utility for ambiguous inputs.
- Blocked inputs are **never added to conversation history**, preventing context pollution for subsequent turns.
- The decision threshold is **configurable** → operators can tune the security/utility balance without retraining.

3.2 Track A Implementation: Decision Engine (decision_engine.py)

Track A implements **Layers 1 and 4** (the filter and gateway) as a fully tested Python module. It is the quantitative backbone of RAGE: all per-turn scores, actions, and audit entries are produced here.

3.2.1 InputFilter — Two-Layer Semantic Guardrail

The InputFilter class implements a two-layer scoring pipeline that always returns an ordinal integer score $\in \{0, 1, 2, 3, 4, 5\}$:

- Layer 1 – Heuristic (always active)**: seven compiled regex patterns covering the threat taxonomy T1–T7 (Table 2). Pattern weights are summed and capped at 5. Zero network dependency; ~microsecond latency.

- **Layer 2 – Semantic LLM (optional, activates when OPENAI_API_KEY is present and heuristic score $\geq$ threshold):** calls gpt-4o-mini with a structured JSON prompt to detect paraphrased or obfuscated attacks. Final score = max(layer1, layer2).

Tag	Threat Category	Score Weight	Example Signal
T1	Role override / instruction reset	+3	"ignore your previous instructions"
T2	Jailbreak / DAN-style	+4	"DAN MODE ACTIVATED", "do anything now"
T3	Fake delimiter injection	+3	"###SYSTEM###", "[INST]", "<>"
T4	Context poisoning	+3	"forget everything you were told"
T5	Persona hijacking	+2	"pretend you are", "act as an uncensored AI"
T6	Exfiltration probe	+3	"reveal your system prompt", "show API keys"
T7	Social engineering escalation	+2	"my CEO authorized this override"

Table 2. Threat taxonomy T1–T7 with heuristic weight contributions.

3.2.2 DecisionGateway — ALLOW / WARN / BLOCK

The DecisionGateway intercepts every user message and applies a strict decision tree based on the score produced by the InputFilter:

Action	Score Range	Behaviour	Principal Agent?
ALLOW	0 – 1	Forwards raw input unchanged. No modifications to context.	■ Sees input
WARN	2 – 3	Logs alert. Injects hidden security reinforcement directive into context. Sanitizes dangerous markers. Forwards augmented input.	■ Sees sanitized + directive
BLOCK	4 – 5	Halts immediately. Returns standardized denial message. Input is NOT added to conversation history.	■ Never sees input

Table 3. Gateway decision tree. WARN preserves utility while reinforcing safety context.

3.2.3 Adversarial Conversation Protocol

The ConversationRunner executes N-turn adversarial scripts and produces a TurnMetric record per turn containing: turn index, raw user input, risk score, action, detection reasons, agent response, and latency. Metrics are exported to metrics.json with a score_vector field ready for sklearn.metrics.roc_auc_score.

4. Results

4.1 Adversarial 3-Turn Conversation Demo

We designed a 3-turn adversarial script with known ground-truth labels (0 = benign, 1 = attack) and ran it through the Track A pipeline in offline mode (heuristic-only, zero API calls).

Turn	Ground Truth	Attack Description	Score	Action	Signals
1	0 – Benign	Legitimate Q2 business query	0/5	ALLOW	None
2	1 – Attack	Multi-vector: role override + DAN jailbreak + fake delimiter injection + context poisoning + persona hijack + exfiltration probe	5/5	BLOCK	T1, T2, T3, T4, T5, T6
3	1 – Attack	Social-engineering escalation + persona hijacking + context poisoning ('I am the CEO, forget your safety guidelines')	5/5	BLOCK	T4, T5

Table 4. Per-turn results from the 3-turn adversarial demo.

4.2 AUC Degradation Metric

Using the per-turn scores as a continuous predictor against binary ground-truth labels:

- **Score vector:** [0, 5, 5]
- **Ground-truth labels:** [0, 1, 1]
- **AUC (sklearn.metrics.roc_auc_score): 1.0000** — perfect separation between benign and attack inputs on this test set.
- **Average risk score:** 3.33 / 5.00
- **Latency per turn (offline/heuristic mode):** ~0.1 ms — negligible overhead.

The score vector [0, 5, 5] shows that the gateway correctly escalated from no risk to maximum risk as soon as adversarial content appeared, with no false positives on the benign turn. The AUC of 1.0 on this controlled dataset validates the heuristic layer's coverage of the T1–T6 attack vectors present in turns 2 and 3.

4.3 ALLOW / WARN / BLOCK Behavior Demonstration

The console output of a full run demonstrates the three gateway actions sequentially:

```
Turn 1 ■ ALLOW ■ Score: 0/5 ■ Input forwarded unchanged to Principal Agent
Response: [Simulated enterprise RAG answer about Q2 sales]

Turn 2 ■ BLOCK ■ Score: 5/5 ■ 6 threat signals detected (T1–T6)
Response: ■ Acceso denegado: Violación de políticas de seguridad
[Input NEVER added to conversation history]

Turn 3 ■ BLOCK ■ Score: 5/5 ■ 2 threat signals detected (T4, T5)
Response: ■ Acceso denegado: Violación de políticas de seguridad
```

Listing 1. Console output excerpt (offline/heuristic mode). Full output and metrics.json available in the repository.

4.4 Cost / Latency Profile

Scenario	Overhead vs base model	Extra latency
Heuristic-only (no LLM layer)	~+0%	~0.1 ms
Hybrid: heuristic + gpt-4o-mini (semantic)	~+1–5%	+200–500 ms
Full pipeline with RAG KB lookup	~+3–12%	+100–400 ms
Worst case (LLM guard as expensive as base)	~+50–100%	+0.5–1 s

Table 5. Estimated cost/latency profile. Blocking attacks early saves downstream LLM calls, partially compensating the overhead.

5. Discussion and Limitations

5.1 Multi-Turn Context Drift — the Gap Guardrails Miss

Single-turn guardrails evaluate each message in isolation. A sophisticated attacker can gradually shift the conversation context across turns — establishing trust in early turns, then injecting the payload in a later turn when defenses are weaker. RAGE's stateful architecture (Layer 3: semantic intent filter with micro-summary) is designed to catch this pattern. Track A already prevents context pollution by not appending blocked inputs to conversation history, preserving the integrity of the trust context for the Principal Agent.

5.2 The WARN Band: Security vs Utility

The WARN band (score 2–3) demonstrates that binary ALLOW/BLOCK is not sufficient: many real-world inputs are ambiguous. The hidden security reinforcement directive injection at WARN level maintains utility (the Principal Agent responds) while reinforcing its operating constraints. This maps to the 'defense in depth' principle: even if the filter is uncertain, the model itself receives explicit re-anchoring.

5.3 Limitations

- **Adaptive attackers:** a motivated attacker who knows the heuristic patterns can craft inputs that score 0–1 by avoiding flagged phrases while preserving malicious intent. The optional LLM semantic layer mitigates this but does not eliminate it.
- **Semantic gap:** the heuristic layer cannot catch 0-day attack variants that do not lexically resemble any T1–T7 pattern. The KB RAG layer (Track B, not yet integrated) is designed to address this.
- **AUC on small samples:** AUC = 1.0 on 3 turns is a proof-of-concept validation, not a production benchmark. Rigorous evaluation requires large, diverse adversarial datasets with balanced class distributions.
- **LLM08 (KB poisoning):** RAGE introduces its own vector store as an attack surface. Indexing content integrity must be controlled (access control + scanning before KB insertion).
- **False positive rate:** the heuristic layer may flag legitimate queries that use natural language resembling threat patterns (e.g., 'How do I ignore previous errors in my code?'). Threshold tuning and per-pattern weight calibration are required in production.
- **Simulated Principal Agent:** the PrincipalAgent in Track A is a deterministic simulator. End-to-end evaluation with a real LLM is planned.

5.4 OWASP LLM Top 10 Coverage

OWASP LLM Risk (2025)	RAGE Coverage	Mechanism
LLM01 Prompt Injection	■ Core (Track A)	4-layer cascade; ALLOW/WARN/BLOCK
LLM07 System Prompt Leakage	■ Core	T6 pattern detects exfiltration probes; canary logging
LLM06 Excessive Agency	■ Planned (Track C)	Tool-call allowlist gating (SELECT only)
LLM02 Sensitive Info Disclosure	■ Planned	Output sanitization layer
LLM05 Improper Output Handling	■ Planned	Output validation + structured response enforcement
LLM08 Vector/Embedding Weaknesses	■■ RAGE introduces risk	Requires KB integrity controls
LLM03/04/09/10	■ Out of scope	Supply chain, training poisoning, misinformation

Table 6. OWASP LLM Top 10 2025 coverage mapping.

6. Conclusion

We presented **RAGE**, a layered defense middleware that protects LLM-based agents against prompt injection, jailbreaks, and multi-turn context manipulation. Our Track A implementation (`decision_engine.py`) delivers a modular, auditable decision engine with a heuristic + semantic dual-layer input filter, a strict ALLOW/WARN/BLOCK gateway, and per-turn metric export ready for AUC-based research evaluation.

RAGE does not claim to solve prompt injection — no defense does. Instead, it occupies a well-defined engineering position: **explainable, model-agnostic, continuously updatable governance** that raises the attacker's cost, maps defenses to an industry-standard taxonomy (OWASP LLM Top 10 2025), and exposes the first standardized multi-turn degradation metric for longitudinal safety research.

The next steps are Track B (threat KB with vector similarity), Track C (stateful intent classifier), and end-to-end integration with a real LLM and a simulated enterprise database to demonstrate DROP TABLE prevention as a concrete measurable outcome. The multi-turn AUC curve — comparing RAGE vs. no defense across adversarial conversation lengths — will be the centerpiece of the quantitative evaluation.

Reproducibility

All code is available at github.com/LuisUPY/RAGE-HACKATHON-PROYECT. The `metrics.json` file generated by each run contains the full `score_vector`, `action_vector`, and per-turn metadata needed to replicate AUC calculations. No API key is required to reproduce the offline (heuristic-only) results.

References

- OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llm-top-10/>
- Nasr, M. et al. (2025). *The Attacker Moves Second: Breaking LLM Defenses with Adaptive Attacks*. OpenAI / Anthropic / Google DeepMind preprint.
- RAD — Retrieval-Augmented Defense against jailbreaks. arXiv:2508.16406 (2025). <https://arxiv.org/abs/2508.16406>
- *Securing AI Agents Against Prompt Injection: A Multi-layer RAG Framework*. arXiv:2511.15759 (2025). 73.2% → 8.7% ASR, 94.3% utility. <https://arxiv.org/abs/2511.15759>
- Chen, S. et al. **StruQ**: Defending Against Prompt Injection with Structured Queries. USENIX Security 2025. <https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe>
- **SecAlign** — Defending Against Prompt Injection with Preference Optimization. arXiv:2410.05451 (2024). ~8% ASR. <https://arxiv.org/abs/2410.05451>
- SISF — Self-Improving Safety Filters (ASR 0.27%). arXiv:2511.07645 (2025). <https://arxiv.org/abs/2511.07645>
- Palit Benchmark — Prompt Injection Trilemma (FP / recall / latency). arXiv:2505.13028 (2025). <https://arxiv.org/abs/2505.13028>
- *Survey: Prompt Injection Attacks on LLMs* (128 studies, 2022–2025). CMC v87n1. <https://www.techscience.com/cmc/v87n1/66084>
- Vigil — LLM Security Scanner. <https://github.com/deadbites/vigil-llm>
- Rebuff — Prompt Injection Detector. <https://github.com/protectai/rebuff>
- ProtectAI LLM Guard. <https://github.com/protectai/llm-guard>
- CaMeL (DeepMind) — Defeating Prompt Injections by Design. 2025.
- RAGE Project Repository. <https://github.com/LuisUPY/RAGE-HACKATHON-PROYECT>

LLM Usage Statement: Large language models (GPT-4o, Claude) were used to assist with literature summarization, code documentation, and initial draft text generation. All architectural decisions, threat taxonomy design, metric definitions, implementation code, and empirical results were independently developed and verified by the team. No claims in this paper rely solely on LLM-generated content.